PRACTICAL WEEK 10 2520090058

1. Investigate inode structures using commands such as ls-i, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

```
  UW PICO 5.09                                File: 3355.c

#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

void show_inode(const char *filename)
{
    struct stat info;

    if (lstat(filename, &info) == -1)
    {
        perror("lstat");
        return;
    }

    printf("\nFile: %s\n", filename);
    printf("Inode number : %llu\n",
           (unsigned long long)info.st_ino);
    printf("File size    : %lld bytes\n",
           (long long)info.st_size);
    printf("Hard links   : %llu\n",
           (unsigned long long)info.st_nlink);
    printf("Permissions  : %o\n",
           (unsigned int)(info.st_mode & 0777));

    if (S_ISLNK(info.st_mode))
        printf("Type         : Symbolic link\n");
    else if (S_ISREG(info.st_mode))
        printf("Type         : Regular file\n");
}

int main(void)
{
    int fd;
    const char *original = "original.txt";
    const char *hard = "hardlink.txt";
    const char *symbolic = "symlink.txt";

    fd = open(original, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    const char *message = "Hello Linux inode!\n";
    if (write(fd, message, strlen(message)) == -1)
    {
        perror("write");
        close(fd);
        return 1;
    }
    close(fd);

    printf("Original file created successfully.\n");

    if (link(original, hard) == -1)
        perror("Creating hard link");
    else
        printf("Hard link created successfully.\n");

    if (symlink(original, symbolic) == -1)
        perror("Creating symbolic link");
    else
        printf("Symbolic link created successfully.\n");

^G Get Help  ^O WriteOut  ^R Read File  ^Y Prev Pg  ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is   ^V Next Pg  ^U UnCut Text ^T To Spell
```

Output:

```
sreenikhilreddy@Srees-MacBook-Pro ~ % nano gf.c
sreenikhilreddy@Srees-MacBook-Pro ~ % nano 3355.c
sreenikhilreddy@Srees-MacBook-Pro ~ %
sreenikhilreddy@Srees-MacBook-Pro ~ % gcc 3355.c -o 3355
sreenikhilreddy@Srees-MacBook-Pro ~ % ./3355
Original file created successfully.
Hard link created successfully.
Symbolic link created successfully.

File: original.txt
Inode number : 21985614
File size    : 19 bytes
Hard links   : 2
Permissions  : 644
Type         : Regular file

File: hardlink.txt
Inode number : 21985614
File size    : 19 bytes
Hard links   : 2
Permissions  : 644
Type         : Regular file

File: symlink.txt
Inode number : 21985617
File size    : 12 bytes
Hard links   : 1
Permissions  : 755
Type         : Symbolic link

Use 'stat -L symlink.txt' to inspect its target.
sreenikhilreddy@Srees-MacBook-Pro ~ %
```

2.Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity. same i have to do in ubuntu. this is practical 10.

```
 UW PICO 5.09                                File: hgf.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main(void)
{
    const char *filename = "mmapfile.txt";
    const char *message = "Hello from mmap in Ubuntu!\n";
    size_t length = strlen(message);

    // Step 1: Create and open the file
    int fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0644);
    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    // Step 2: Set the file size before mapping
    if (ftruncate(fd, length) == -1)
    {
        perror("ftruncate");
        close(fd);
        return 1;
    }

    // Step 3: Map the file into memory
    char *ptr = mmap(NULL, length, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);

    if (ptr == MAP_FAILED)
    {
        perror("mmap");
        close(fd);
        return 1;
    }

    // Step 4: Write data through the mapped memory
    memcpy(ptr, message, length);

    // Step 5: Read and display mapped data
    printf("Data read using mmap():\n%s", ptr);

    // Step 6: Modify data in mapped memory
    const char *updated = "Updated using mmap in Ubuntu!\n";
    size_t updated_len = strlen(updated);

    if (updated_len <= length)
    {
        memcpy(ptr, updated, updated_len);
        if (updated_len < length)
            memset(ptr + updated_len, ' ', length - updated_len);

        printf("\nModified data using mmap():\n");
        fwrite(ptr, 1, length, stdout);
        printf("\n");
    }
    else
    {
        printf("Updated text is too long for the mapped region.\n");
    }

    // Step 7: Synchronize mapped changes with the file
    if (msync(ptr, length, MS_SYNC) == -1)

^G Get Help      ^O WriteOut      ^R Read File     ^Y Prev Pg       ^K Cut Text      ^C Cur Pos
^X Exit          ^J Justify       ^W Where is      ^V Next Pg       ^U UnCut Text    ^T To Spell
```

OUTPUT:

```
sreenikhilreddy@Srees-MacBook-Pro ~ % nano hgf.c
sreenikhilreddy@Srees-MacBook-Pro ~ % gcc hgf.c -o hgf
sreenikhilreddy@Srees-MacBook-Pro ~ % ./hgf
Data read using mmap():
Hello from mmap in Ubuntu!
Updated text is too long for the mapped region.

File operation completed successfully.
sreenikhilreddy@Srees-MacBook-Pro ~ %
```